

Introduction:

SVM which stands for Support vector machines is a supervised machine learning algorithm which solves classification and regression tasks. More specifically, it is used to find the hyperplane that maximises the margin between two classes; Margin being the distance between the hyperplane and the closest points from each class. SVM was introduced to solve the limitations of earlier classifiers which struggled with data which is not linearly separable. It uses kernel functions which map data into higher dimensional spaces where separation is easier. SVM has developed significantly over time with polynomial kernels which made SVM highly versatile and even later SVR (support vector regression).

A neural network is a machine learning model which consists of layers of connected nodes called neurons which process inputs through weighted connections and activation functions. Neural networks were also used to overcome the limitations of simple classifiers which struggle with nonlinear decision boundaries in data. Initially, early models were like the perceptron which could only solve linear problems. MLPs were then later developed and back-propagation which expanded their capabilities to learn nonlinear decision boundaries. Backpropagation allowed for gradients to be computed through layers and made it possible to adjust weights and train deeper networks. The MLP trained with back propagation still faced some barriers such as computational power and data availability. As a result, the rise of deep learning came through where they had access to large datasets, powerful GPUs and improved training techniques such as ReLU activations which allowed for training of very deep networks i.e.CNNs which make up the backbone of a lot of the current artificial intelligence.

It is particularly meaningful to compare SVMs and neural networks as they represent two distinct approaches to supervised learning. SVMs are often used for small to medium sized datasets with high dimensional feature space at a low computation cost while Neural networks by contrast, are exceptional with large-scale data. Comparing the two; in particular their hyperparameters can allow for researchers to understand when to use a SVM model and to use a neural network. This comparison is still relevant today as choosing the model depends on many things such as accuracy but also data availability and computational efficiency.

Method:

To compare SVM and neural networks, two different methods will be deployed for each and to be analysed using graphs.

SVM method:

1. Data preparation: loaded the training data set (train.csv), split into 4000 samples for training and 4500 for validation. Labels were also converted from $\{0,1\}$ to $\{-1, +1\}$ and standardized features using the training mean and standard deviation.
2. Formulation of the optimization problem: Defined the primal objective function:

$$\min_{w,b,\xi} \frac{1}{2} ||w||^2 C \sum_{i=1}^n \xi_i$$

Subject to:

$$y_i(W^T x_i + b) \geq 1 - \xi_i, \xi_i \geq 0$$

Constructed the matrices P, q, G, h to represent this as a QP problem, solved using the [cvxopt.solvers.qp](https://cvxopt.org/solvers/qp) function to obtain w,b and the slack variable.

3. Validation: Predicted labels set using $\hat{y} = \text{sign}(w^T x + b)$ and compute the validation accuracy by comparing the predicted labels with the true labels.
4. Hyperparameter Tuning: Performed grid search over different values of C = 0.0001, 0.0003, 0.001, 0.003, 0.01, 0.03, 0.1, 0.3, 1.0, for each C trained the model and evaluated validation accuracy, selecting the C with the highest validation performance
5. Final testing: Retrained model on best C value and standardized the test data. Predicted labels on the test set and reported final classification accuracy.

Neural networks method:

1. Data preparation: similarly to SVM, loaded the training dataset train.csv and split into 400 samples for training. Convert the labels to integers and one-hot encoded them for the cross entropy loss.
2. Architecture: Implemented a multi-layer perceptron with input size equal to the number of features, one hidden layer with 64 neurons and ReLU activation. The forward pass can be described by this sequence:

Input -> linear -> ReLU -> linear -> output logits

3. Objective (Loss function): used cross entropy loss which measures the difference between the predicted class probabilities and the true labels: averages over the batch and uses one-hot encoded labels.

$$L_{batch} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^K y_{i,c} \log(\hat{y}_{i,c})$$

4. Optimizer and Hyperparameters: optimiser used is SGD with L2 weight decay, and hyperparameters include learning rate, batch size, number of hidden units and number of epochs.
5. Training procedure: for each epoch, shuffled the training data and created minibatches. Afterwards, computed the forward pass to get the predictions and loss then do backpropagation to compute gradients, updating the weights using SGD with L2 regularisation. Track and record the training loss and accuracy for every 10 epochs.
6. Hyperparameter tuning: hidden layer size, learning rate and weight decay were chosen based on an experimental search process. Different candidate values were tested for each hyper parameter and the models were trained using the training set. Selection of hyperparameters were based on Training accuracy as well as validation performance to avoid overfitting(if the difference was too great)
7. After training, predictions were obtained and accuracy was computed as the proportion of the correctly classified samples in the validation set

8. The final model is used on a testing set to obtain final prediction accuracy on unseen data.

Experimental setup:

Dataset: experiments were performed on the given train.csv dataset and final evaluation was performed on the test.csv dataset. The dataset consists of input features and binary class labels (0 and 1). The first 4000 samples of the training data were used for training and the remaining 4500 for validation.

Preprocessing: for SVM, labels were mapped from 0,1 to -1,+1, and features were standardized using the training mean and standard deviation. For neural networks, labels were converted to integers and one-hot encoded for the cross entropy loss. Features were normalized the same way as SVM was.

Evaluation metric: Model performance was evaluated using classification accuracy, which is simply the proportion of correctly predicted labels with respect to the total number of samples.

SVM Results: Hyperparameter tuning of regularisation parameter C:

C value	Training accuracy	Validation Accuracy (%)
0.0001	0.8792	0.9144
0.0003	0.9473	0.9358
0.001	0.9670	0.9153
0.003	0.9708	0.9058
0.01	0.9765	0.9027
0.03	0.9768	0.8996
0.1	0.9800	0.8951
0.3	0.9815	0.8942
1.0	0.9818	0.8942

Best C	Test accuracy (%)
0.0003	0.934

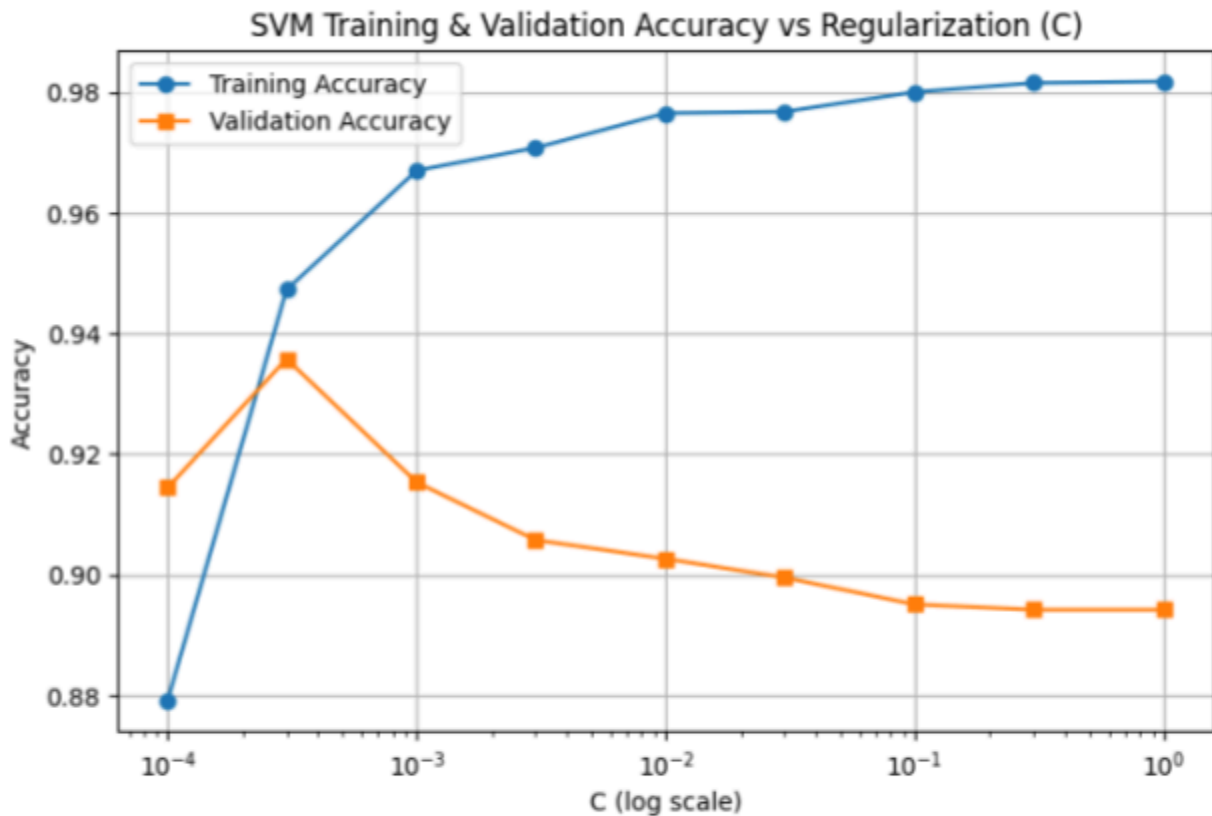


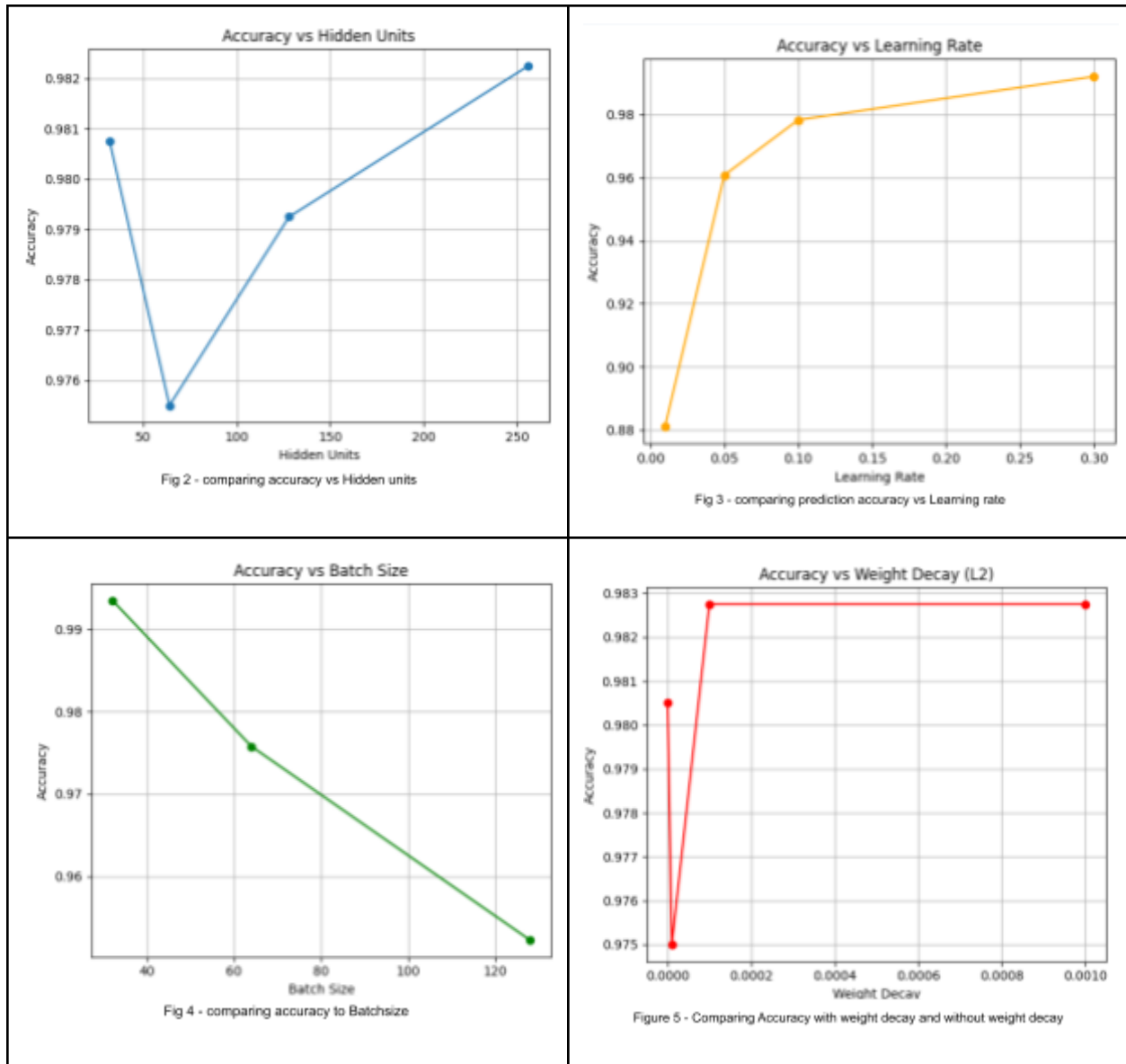
Fig 1 - line graph showing difference in training and validation accuracy to increasing C value

SVM analysis: C value for SVM determines how much penalty is applied to misclassified points. As can be seen from figure 1 and the table for validation accuracy, the peak accuracy was around $C = 0.0003$ with a 0.933 accuracy, which suggests that a small regularisation penalty works best for this dataset. At the smallest C value, the accuracy is slightly lower at 0.914, indicating slight underfitting as the model allows more slack for misclassifications. This makes sense as a small C value does not punish errors very heavily and thus points which are extremely hard to classify such as potential outliers would be ignored and thus the accuracy isn't 100 percent. For moderate to high C values the accuracy gradually decreases, reflecting that the model starts to overfit by attempting to classify every point correctly. This is reaffirmed by the significant difference between the high training accuracy and its validation accuracy as can be seen in fig1 (after c-value 0.0003). At very high C values, you can see that the graph begins to plateau at 0.895 showing that the further increasing C does not improve validation performance and may even slightly reduce generalisation. This can be explained by the fact that a large C value means being extremely strict about misclassification so the SVM tries to adjust the hyperplane to correctly classify every single point which results in the decision boundary or the hyperplane bending in a way that no longer represents the majority of the points well and can cut into clusters of non-outlier points. This will in turn cause the drop in validation accuracy. Since a low C value of 0.0003 worked best for this dataset it suggests that the model works best

when capturing the main patterns and ignoring minor deviations that don't represent the general trend. It is clearly evident that the model is most successful when its training accuracy is close to its validation accuracy.

Neural networks Results:

Hyperparameter Experiments:



Looking at some of the hyper parameter changes: For hidden units there is no general trend as accuracy does not increase steadily with hidden units, instead it drops from 32 to 64 hidden units and then rises again up to 256. Hidden units transform the input data so that it's easier for the network to solve the task. When there are too little hidden units, the network may not have enough capacity to learn the full patterns and will therefore be underfitting. When there are too many hidden units, the network will become too complex and start memorising instead of

generalising. This can be observed in fig2 as the accuracy for when the hidden units is 256 is really high and since the validation accuracy is much higher than training accuracy, it would be overfitting at 256 units. For learning rate, accuracy increases sharply from approximately 0.01 to 0.05, and then slowly increases from 0.05 to 0.30. This is expected as a very small learning rate means that the network will update the weights very slowly, which leads to a slow convergence, thus in the 50 epochs, 0.01 learning rate resulted in the worst accuracy. Once the learning rate is large enough to allow for effective learning, you can see that the accuracy substantially increases and increasing it further has diminishing returns. In terms of batch size, accuracy decreases substantially as batch size increases. Batch size controls how many samples are processed at once before the model updates its weight. The smaller the batch size, the more weight updates made per epoch which correlates to fig4 as increasing the batch size decreases the accuracy due to inaccurate weights of the variables. In terms of the L2 regularisation, it can be seen that without weight decay it fits data pretty well with an accuracy of around 0.980. The shape of the graph appears to look like a square root, with a sharp dip from L2 value of 0 to $1e-5$ and then plateaus after 0.0001. A justification for this could be that this really small L2 value isn't enough to improve generalisation. If the decay value is too small it most likely doesn't make a big enough impact to fit the training data properly resulting in a dip in accuracy. However, there is a really big improvement in accuracy at $1e-4$ suggesting that the regularisation constant finally becomes effective. The accuracy then remains constant as the model reaches a balance between fitting new data and generalising unseen data.

Results of Best selection of hyper parameters:

Based on the trends observed in fig 2-5, the most optimal Hyperparameters were chosen as the final NN-model.

HyperParameters chosen: hidden_dim=32, lr=0.3, batch_size=32, weight_decay= $1e-4$			
Epoch	Training Accuracy	Validation accuracy	Loss
10	0.9725	0.9418	0.120233
20	0.9935	0.9509	0.060740
30	0.9978	0.9538	0.037099
40	0.9992	0.9569	0.025524
50	0.9998	0.9582	0.019226
Final test accuracy	0.9673		

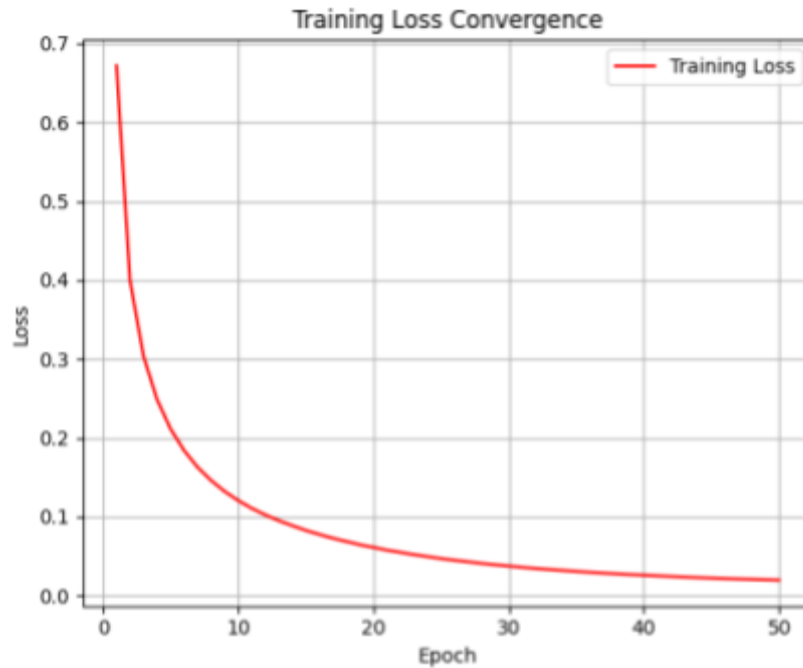


Fig 6 - Converge of Loss graph for Neural network model

Performance of the best model in Pytorch:

To evaluate how effective the custom implementation is, a comparison against a pytorch implementation using the same dataset and hyper parameters is performed. This is used as a benchmark to determine whether my model achieves similar accuracy.

HyperParameters chosen: hidden_dim=32, lr=0.3, batch_size=32, weight_decay=1e-4

Epoch	Training Accuracy	Validation accuracy	Loss
10	1.0	0.9853	0.0011
20	1.0	0.9858	0.0005
30	1.0	0.9860	0.0004
40	1.0	0.9864	0.0003
50	1.0	0.9869	0.0003
Final test accuracy	0.9880		

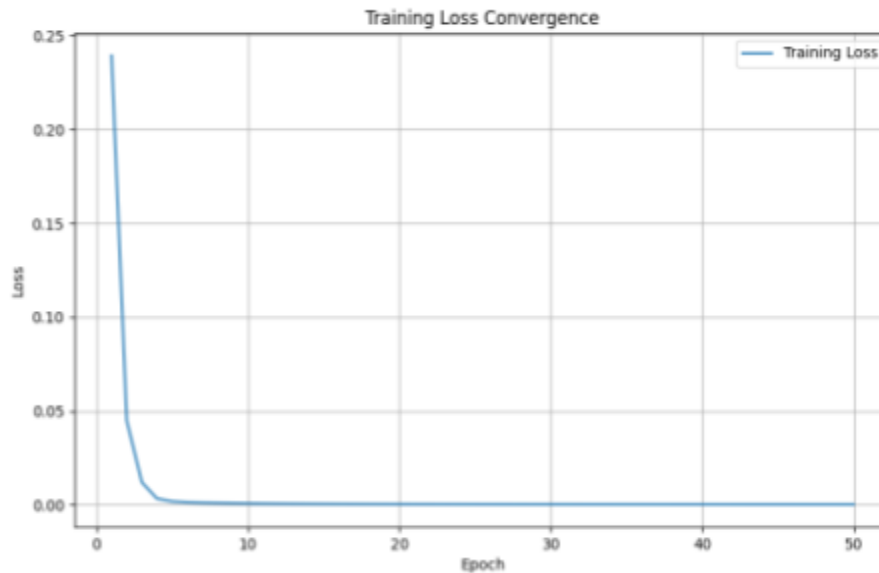


Fig 7 - Loss convergence graph for pytorch implementation of NN-model

Comparison of SVM, Custom-NN and Pytorch-NN

Looking at all three model's performance, SVM achieved the lowest accuracy of 93.4%, the custom NN performed better than SVM with an accuracy of 96.7%. The Pytorch NN slightly outperformed the custom implementation with an accuracy of 98.8%. A likely reason as to why neural network model worked better for this data set than the SVM did is that neural networks are able to model complex non-linear relationships in the data through their hidden layers and the non-linear activation functions, contrasting to SVM especially linear kernel are more limited in capturing these patterns. The hidden units in the Neural network allowed the model to learn intermediates features and improve to generalise unseen data better. Pytorch NN not only had very high training accuracy but it also had minimal drop in validation showing effective optimization throughout and consistency.

In terms of training vs validation behaviour, it is evident that for SVM, when the C value is relatively small, the training and validation accuracies are relatively close which means less overfitting. Contrastingly in the custom neural network, even though it displayed higher training accuracy and good validation accuracy, the difference between the training accuracy and validation accuracy was still greater showing the higher model capacity and the complexity of the neural network but also slightly more overfitting.

Comparing the custom neural network model with the pytorch-NN, it is also evident that the loss at every 10 epochs was significantly less in the pytorch-NN compared to my model which suggests that it optimized the parameters more effectively and also converged faster than my model. This is also justified by the loss convergence graph in figure 6 and 7 showing that the Pytorch implementation converges faster shown by a much steeper curve at the beginning of the graph. A plausible justification for why Pytorch NN has lower loss and converges faster is that PyTorch is highly optimised. A lot of 'under the hood' implementations are probably faster

than my implementations such as python loops and the manual computations which I did to calculate derivatives, which may slow down convergence.

Overall, these results are consistent with theoretical expectations as neural networks outperform SVM due to their ability to model non-linear patterns, hyperparameters like hidden units, learning rate, and C value influence both training and generalization performance, and optimised implementations like pytorch has shown to allow even faster convergence at a lower loss.

conclusion

The study compared Support vector machines and neural networks on a binary classification task. The results demonstrated that while SVM demonstrated really high accuracy with an optimal regularisation parameter $C = 0.0003$, its test accuracy 93.4% was lower than both the custom neural network(96.7%) and the Pytorch neural network (98.8%). The neural networks were able to capture complex non-linear relationships through their hidden representations, resulting in better generalisation on unseen data. The SVM exhibited a clear trade off between bias and variance, that being small values of C avoided overfitting but risked underfitting whereas larger values of C improved the training accuracy but led to a significantly lower validation performance. The neural networks, while more flexible, showed signs of overfitting at higher hidden units and when there is a lack of regularisation. Pytorch implementation significantly outperformed the custom model due to the optimised computational routines and much faster convergence speed, as seen by the much lower loss. Overall, SVM is still a viable option for smaller datasets and when efficiency are priorities. Neural networks contrastingly are more suitable for complex and large datasets and when the decision boundaries are more complex. This comparison has highlighted the importance of both models and why careful hyperparameter tuning is crucial and directly correlated with each of the model's performance.